



Prerequisites

CEN207 Data Structures — what to bring on day one

Asst. Prof. Dr. Uğur CORUH · Fall 2026-2027

Why this matters from week 1

- From the first week you **write, build and test programs on your own computer**
- First in **C** for the midterm project, later in **Java** for the final project
- You keep your work under **Git and GitHub** throughout
- You therefore need to arrive with the knowledge and tools in this deck

Formal prerequisites

- CEN107 Algorithms and Programming I
- CEN108 Algorithms and Programming II

The development environment, Git, unit testing and template usage taught in CEN107's first weeks are **assumed** in this course.

Before the first class: work through the self-check tasks near the end of this deck, on your own computer.

1. What we bring from CEN107/CEN108

Development environment

- Compiler (GCC/Clang/MSVC), IDE, WSL on Windows, building with CMake
- Taught in: **CEN107 Week 2 — Development environments**
- Used here in: week 1's C workshop; setting up the midterm (C) project

Git and GitHub

- Creating a repository, `clone`, `commit`, branches, `pull request`, `.gitignore`
- Taught in: **CEN107 Week 3 — Version management with Git**
- Used here in: the project guide's workflow (fork, plan, submission) for both checkpoints

Unit testing and coverage tools

- Writing tests, running them, reading a coverage report
- Taught in: **CEN107 Week 4 — Unit testing and libraries**
- Used here in: the midterm rubric (GoogleTest, gcov/lcov) and the final rubric (JUnit 5, JaCoCo)

Using project templates

- Forking a template, building it, producing its tests and documentation
- Taught in: **CEN107 weeks 2–4**
- Used here in: the project guide's `cpp-cmake-ctest-template` (midterm) and `eclipse-java-maven-template` (final)

Basic algorithm analysis and recursion

- Counting operations, writing a simple recursive function
- Taught in: **CEN108, early weeks**
- Used here in: the Big-O recap in week 1; recursive solutions from week 3 onward (Towers of Hanoi, DFS, tree traversals, merge sort, quicksort)

2. C and Java programming fundamentals

C fundamentals (midterm project)

- Pointers and pointer arithmetic (`int *p;` , `p + 1` , `p` vs. `*p`)
- Arrays: declaration, indexing, passing to a function
- `struct` definitions; grouping fields (e.g. a linked-list node)
- Dynamic memory: `malloc` / `free` , and why every `malloc` needs a matching `free`
- Recursion: a function that calls itself, with a base case
- File I/O, including **binary files** (`fopen` with `"rb"` / `"wb"` , `fread` , `fwrite`)

Java fundamentals (final project)

- Classes and objects: fields, constructors, methods, `this`
- Generics basics: using `List<T>`, declaring a simple generic class or method
- Exceptions: `try` / `catch` / `finally`, checked vs. unchecked

3. Background briefly recalled in class

It is fine not to remember these in full detail — week 1 gives a short recap before they are needed.

Background	Needed in which week?
Big-O notation: counting operations, comparing growth rates	Week 1, then every week to compare data structures
Memory layout: stack vs. heap	Week 1, then whenever pointers and dynamic memory are used



4. What must be installed

Required toolchain

- A C compiler: **Visual Studio 2022** (Desktop dev with C++) on Windows, or **GCC/Clang** on Linux/WSL/macOS
- **CMake** — the midterm template is built and tested with it
- A C coverage tool: **gcov/lcov** on Linux/WSL, or **OpenCppCoverage** with MSVC on Windows
- **Doxygen** — generates the project documentation
- **JDK 21** and **Maven** — Maven pulls in JUnit 5 and JaCoCo automatically
- **Git** and a **GitHub** account

Setup steps and the project templates themselves: the **project guide**.



5. Test yourself before the first class

Task 1 — Compile and predict

What does this program print?

```
int x = 5;  
int *p = &x;  
*p = *p + 1;  
printf("%d %d\n", x, *p);
```

Task 1 — Answer

6 6

`p` holds the address of `x`, so `*p` and `x` name the same memory location; changing one changes the other.

Task 2 — Pointer arithmetic

Given `int arr[4] = {10, 20, 30, 40}; int *p = arr;`, what are the values of `*(p + 2)` and `p[2]` ?

Task 2 — Answer

Both are `30`. `p[2]` is defined to mean `*(p + 2)` — array indexing is pointer arithmetic written with different syntax.

Task 3 — Find the bug

```
int *make_array(int n) {  
    int arr[n];  
    for (int i = 0; i < n; i++) arr[i] = i * i;  
    return arr;  
}
```

What is wrong, and how would you fix it?

Task 3 — Answer

`arr` is a local array on the **stack**; it is destroyed when the function returns, so the returned pointer is **dangling**. Fix: allocate on the **heap** instead —

```
int *arr = malloc(n * sizeof(int));
```

— and let the caller `free` it once done.

Task 4 — Struct and recursion

Define `struct Node { int value; struct Node *next; };` and write a recursive `int length(struct Node *head)` that counts the nodes.

What does `length(NULL)` return, and why does the recursion stop there?

Task 4 — Answer

```
int length(struct Node *head) {  
    if (head == NULL) return 0;  
    return 1 + length(head->next);  
}
```

`length(NULL)` returns `0` — the **base case**; without it the recursion never stops.

Task 5 — Binary file I/O

```
int values[3] = {1, 2, 3};  
FILE *f = fopen("data.bin", "wb");  
fwrite(values, sizeof(int), 3, f);  
fclose(f);
```

How many bytes does `data.bin` contain, and why?

Task 5 — Answer

`3 * sizeof(int)` bytes — **12 bytes** on almost every current desktop platform. `fwrite` copies the raw bytes of the array into the file; unlike `fprintf`, there is no text formatting involved.

Task 6 — Java generics

```
List<Integer> numbers = new ArrayList<>();  
numbers.add(10);  
numbers.add(20);  
int sum = 0;  
for (int n : numbers) { sum += n; }  
System.out.println(sum);
```

What does this print?

Task 6 — Answer

30

`List<Integer>` is a generic collection restricted to `Integer` elements; the for-each loop reads every element and accumulates it into `sum`.

Task 7 — Java exceptions

```
try {  
    System.out.println("A");  
    throw new RuntimeException("boom");  
} catch (RuntimeException e) {  
    System.out.println("B");  
} finally {  
    System.out.println("C");  
}
```

In what order do the three letters print?

Task 7 — Answer

A
B
C

The exception is caught immediately by the matching `catch ( B ); finally` always runs afterward, whether or not an exception was thrown (`C`).

Task 8 — Build and test both toolchains

C toolchain: `gcc --version` · `cmake --version` · `clone`
`cpp-cmake-ctest-template` · `cmake -S . -B build && cmake --build build` ·
`ctest --test-dir build`

Java toolchain: `java -version` · `mvn -version` · `clone`
`eclipse-java-maven-template` · `mvn test`

Task 8 — What success looks like

- `ctest` : 100% tests passed
- Maven: **BUILD SUCCESS**, Failures: 0
- No compiler errors along the way in either toolchain

If either one fails: go back to "What must be installed," fix the missing tool, and try again before the first class.

Ready for the course

If all eight tasks made sense and both toolchains built and tested cleanly, you are ready.

Next: the **syllabus** (weekly schedule) and the **project guide** (exact rubric for each checkpoint).